

PYTHON + PYGAME ZERO

Do zero ao primeiro jogo

Um guia prático para quem está começando em programação e quer aprender criando jogos.

Objetivo: aprender fundamentos de Python e usar Pygame Zero para criar jogos 2D simples, entendendo cada parte do código.

Baseado na documentação oficial do Pygame Zero 1.2.1.

Como usar este guia

O melhor jeito de aprender programação é alternar leitura e prática. Em cada capítulo, leia a explicação, digite os exemplos por conta própria e só depois faça os exercícios. Evite apenas copiar e colar: digitar o código ajuda a perceber nomes, símbolos e indentação.

- Leia um tópico por vez.
- Digite os exemplos e execute-os.
- Altere valores para observar o que muda.
- Faça os exercícios sem olhar a solução.
- Quando algo der erro, leia a mensagem de erro antes de apagar o código.

Regra de ouro: Se um exercício parecer difícil, divida-o em partes menores. Primeiro faça funcionar; depois melhore.

Roteiro de estudo

Parte	Tema	Você vai aprender
Parte 1	Python essencial	Variáveis, tipos, entrada/saída, condições, repetição, funções e listas.
Parte 2	Primeiros passos com Pygame Zero	Instalação, estrutura do arquivo, tela, draw() e update().
Parte 3	Jogos 2D na prática	Teclado, mouse, Actors, movimento, colisões, som e tempo.
Parte 4	Projeto final	Um jogo completo simples para juntar os conceitos.
Parte 5	Próximos passos	Ideias de evolução e desafios extras.

Parte 1 - Fundamentos de Python

Antes de criar jogos, é importante dominar alguns conceitos básicos. Você não precisa aprender “tudo” de Python primeiro; basta construir uma base sólida e ir ampliando conforme os projetos exigirem.

1. Seu primeiro programa

Python executa instruções de cima para baixo. A função `print()` mostra informações no terminal.

```
print("Olá, mundo!")  
print("Estou aprendendo Python para criar jogos!")
```

Dica: Textos ficam entre aspas. Em Python, eles são chamados de strings.

Comentários

Comentários são anotações para humanos. Python ignora tudo que vem depois de `#` na mesma linha.

```
# Este é um comentário  
print("Olá!") # comentário no fim da linha
```

Exercício 1

Crie um programa que mostre seu nome, seu jogo favorito e uma frase dizendo por que você quer aprender programação.

2. Variáveis e tipos de dados

Variáveis guardam valores para que possamos reutilizá-los e modificá-los durante o programa.

```
nome = "Alex"  
idade = 16  
vida = 100  
velocidade = 4.5  
vivo = True  
  
print(nome)  
print(vida)
```

Tipo	Exemplo	Uso comum em jogos
str	"Jogador"	Nome, mensagens, caminhos de arquivos
int	100	Vida, pontos, moedas, fase
float	3.5	Velocidade, tempo, posição

		mais precisa
bool	True / False	Vivo/morto, pausado, venceu/perdeu

Operações

```

vida = 100
vida = vida - 25
moedas = 10
moedas += 5

print(vida) # 75
print(moedas) # 15

```

Exercício 2

Crie as variáveis nome_jogador, vida, dano e moedas. Simule um ataque que reduz a vida e uma recompensa que aumenta as moedas. Mostre o resultado na tela.

3. Entrada de dados e conversão

A função input() permite receber texto digitado pelo usuário. Como input() sempre retorna texto, usamos int() ou float() quando precisamos de números.

```

nome = input("Qual é o seu nome? ")
idade = int(input("Qual é a sua idade? "))

print("Olá,", nome)
print("No ano que vem você terá", idade + 1, "anos.")

```

Dica: Se o usuário digitar letras onde int() espera um número, ocorrerá um erro. Mais tarde você pode aprender try/except para tratar isso.

4. Decisões com if, elif e else

Jogos tomam decisões o tempo todo: “se a vida chegou a zero”, “se encostou no inimigo”, “se apertou uma tecla”. Em Python usamos if.

```

vida = 20

if vida <= 0:
    print("Game Over")
elif vida <= 30:
    print("Cuidado! Vida baixa.")
else:
    print("Continue lutando!")

```

A indentação é parte da sintaxe do Python. O código que pertence ao if precisa ficar recuado, normalmente com 4 espaços.

Comparações importantes

```
x == 10 # igual
x != 10 # diferente
x > 10 # maior
x < 10 # menor
x >= 10 # maior ou igual
x <= 10 # menor ou igual
```

Exercício 3

Crie um sistema de porta: se o jogador tiver uma chave, mostre “Porta aberta”; caso contrário, mostre “Você precisa encontrar a chave”.

5. Repetição com for e while

Laços permitem repetir ações. Isso é útil para contagens, listas de inimigos e processos que se repetem.

```
for numero in range(5):
    print("Onda", numero + 1)

energia = 3

while energia > 0:
    print("Correndo...")
    energia -= 1

print("Sem energia!")
```

Dica: Em jogos gráficos, normalmente você não cria um while infinito por conta própria. O Pygame Zero já possui um loop do jogo e chama `update()` repetidamente.

6. Funções

Funções agrupam comportamentos. Elas ajudam a evitar repetição e deixam o código organizado.

```
def causar_dano(vida, dano):
    vida -= dano
    return vida

vida_jogador = 100
vida_jogador = causar_dano(vida_jogador, 25)
print(vida_jogador)
```

Exercício 4

Crie uma função curar(vida, quantidade) que devolva a nova vida. Depois teste com valores diferentes.

7. Listas e aleatoriedade

Listas guardam vários valores. Em jogos elas são úteis para inimigos, itens, fases e posições.

```
inimigos = ["slime", "morcego", "esqueleto"]

print(inimigos[0])

for inimigo in inimigos:
    print("Encontrou:", inimigo)
```

O módulo random permite criar comportamentos aleatórios.

```
import random

numero = random.randint(1, 6)
print("Dado:", numero)

inimigo = random.choice(["slime", "morcego", "fantasma"])
print("Inimigo:", inimigo)
```

8. Mini desafio de Python: batalha no terminal

Antes de entrar no Pygame Zero, tente construir uma batalha simples. O jogador e o inimigo começam com vida e trocam ataques até alguém chegar a zero.

```
import random

vida_jogador = 100
vida_inimigo = 60

while vida_jogador > 0 and vida_inimigo > 0:
    dano = random.randint(8, 18)
    vida_inimigo -= dano
    print("Você causou", dano, "de dano.")

    if vida_inimigo <= 0:
        break

    dano_inimigo = random.randint(5, 15)
    vida_jogador -= dano_inimigo
    print("O inimigo causou", dano_inimigo, "de dano.")
```

```
print("Sua vida:", vida_jogador)  
print("Vida do inimigo:", vida_inimigo)
```

Dica: Tente adicionar poção, defesa, ataque crítico e escolha de ações. Esse tipo de melhoria é excelente prática.

Parte 2 - Começando com Pygame Zero

Pygame Zero foi criado para simplificar o desenvolvimento de jogos 2D em Python, removendo parte do código de configuração necessário em bibliotecas de jogos mais tradicionais. Ele é ótimo para aprender os fundamentos de lógica de jogos.

9. Instalação

Você precisa ter Python instalado. Em seguida, abra o terminal ou prompt de comando e instale o Pygame Zero:

```
pip install pgzero
```

Crie um arquivo chamado, por exemplo, jogo.py. Para executar:

```
pgzrun jogo.py
```

Dica: Se o comando `pgzrun` não for reconhecido, confirme se o pacote foi instalado no mesmo Python usado pelo terminal/IDE.

10. Estrutura mínima de um jogo

Um programa Pygame Zero pode definir o tamanho da janela e uma função `draw()`, que é chamada pelo framework para desenhar a tela.

```
WIDTH = 800
HEIGHT = 600

def draw():
    screen.fill("black")
    screen.draw.text("Meu primeiro jogo", center=(400, 300), fontsize=50, color="white")
```

Ao executar com `pgzrun`, uma janela de 800 x 600 pixels será aberta.

Coordenadas da tela

Em jogos 2D, `x` cresce da esquerda para a direita. `y` cresce de cima para baixo. O ponto (0, 0) fica no canto superior esquerdo.

```
(0, 0) -----> x
|
|
|
v
y
```


11. draw() e update()

A função `draw()` deve desenhar o estado atual do jogo. A função `update()` deve alterar o estado: mover personagens, verificar regras e atualizar valores. A documentação oficial destaca essa separação.

```
x = 100

def update():
    global x
    x += 2

def draw():
    screen.fill("black")
    screen.draw.filled_circle((x, 300), 20, "yellow")
```

Dica: Pygame Zero chama `update()` repetidamente, normalmente cerca de 60 vezes por segundo. Evite colocar movimento dentro de `draw()`.

12. Formas e texto

```
WIDTH = 800
HEIGHT = 600

def draw():
    screen.fill("navy")
    screen.draw.line((50, 50), (300, 50), "white")
    screen.draw.rect(Rect((100, 100), (200, 80)), "orange")
    screen.draw.filled_circle((400, 250), 40, "red")
    screen.draw.text("Pontuação: 0", (20, 20), fontsize=32, color="white")
```

Exercício 5

Faça uma tela com fundo escuro, um retângulo representando uma plataforma, um círculo representando o jogador e um texto com “Fase 1”.

Parte 3 - Construindo jogos 2D

13. Movimento com teclado

O objeto `keyboard` permite verificar se uma tecla está pressionada.

```
x = 400
y = 300
velocidade = 4

def update():
    global x, y

    if keyboard.left:
        x -= velocidade
    if keyboard.right:
        x += velocidade
    if keyboard.up:
        y -= velocidade
    if keyboard.down:
        y += velocidade

def draw():
    screen.fill("black")
    screen.draw.filled_circle((x, y), 20, "lime")
```

Limites da tela

```
x = max(20, min(WIDTH - 20, x))
y = max(20, min(HEIGHT - 20, y))
```

Esse padrão impede que o centro do jogador atravessasse os limites definidos.

14. Eventos de teclado e mouse

Além de verificar teclas continuamente, você pode reagir a eventos específicos.

```
def on_key_down(key):
    if key == keys.SPACE:
        print("Espaço pressionado!")

def on_mouse_down(pos, button):
    print("Clique em:", pos)
```

Eventos são úteis para tiros, menus, botões e ações que acontecem uma vez por pressionamento.

15. Actors: personagens com imagem e posição

Actor é um dos recursos mais importantes do Pygame Zero. Ele representa uma imagem posicionada na tela e fornece propriedades úteis de posição e colisão.

Crie uma pasta images ao lado do arquivo .py e coloque dentro dela uma imagem, por exemplo player.png. No código, use o nome sem a extensão:

```
player = Actor("player", (400, 300))

def draw():
    screen.clear()
    player.draw()
```

Movendo um Actor

```
def update():
    if keyboard.left:
        player.x -= 4
    if keyboard.right:
        player.x += 4
    if keyboard.up:
        player.y -= 4
    if keyboard.down:
        player.y += 4
```

Dica: Actor possui propriedades como *x*, *y*, *pos*, *left*, *right*, *top*, *bottom* e *center*, que facilitam posicionamento e limites.

16. Colisões

Actors podem verificar colisões com outros objetos. Para dois Actors, uma opção simples é `colliderect()`.

```
player = Actor("player", (100, 300))
inimigo = Actor("enemy", (600, 300))

def update():
    if keyboard.right:
        player.x += 4

    if player.colliderect(inimigo):
        print("Colisão!")

def draw():
    screen.clear()
    player.draw()
    inimigo.draw()
```

Distância como alternativa

Em jogos com objetos aproximadamente circulares, você também pode comparar a distância entre os centros.

```
if player.distance_to(inimigo) < 50:
    print("Muito perto!")
```

17. Listas de inimigos

Um jogo raramente tem um único inimigo. Listas permitem controlar vários Actors.

```
inimigos = [
    Actor("enemy", (200, 100)),
    Actor("enemy", (500, 200)),
    Actor("enemy", (650, 450)),
]

def draw():
    screen.clear()
    for inimigo in inimigos:
        inimigo.draw()
```

Exercício 6

Crie três inimigos em posições diferentes. Faça o jogador mover com as setas e exiba “Game Over” quando encostar em qualquer inimigo.

18. Pontuação e coleta de itens

```
pontos = 0
moeda = Actor("coin", (500, 300))

def update():
    global pontos

    if player.colliderect(moeda):
        pontos += 1
        moeda.pos = (100, 100)

def draw():
    screen.clear()
    player.draw()
    moeda.draw()
    screen.draw.text(f"Pontos: {pontos}", (20, 20), fontsize=36, color="white")
```

Dica: Depois você pode mover a moeda para uma posição aleatória usando `random.randint()`.

19. Sons e música

Pygame Zero reconhece pastas de recursos. Sons normalmente ficam em sounds e músicas em music.

```
def on_key_down(key):  
    if key == keys.SPACE:  
        sounds.laser.play()
```

Para música de fundo:

```
music.play("tema")  
music.set_volume(0.5)
```

Dica: Use arquivos em formatos compatíveis e nomes simples, sem espaços, para reduzir problemas com caminhos.

20. Tempo e Clock

O clock do Pygame Zero permite agendar funções para executar no futuro ou em intervalos.

```
def criar_inimigo():  
    print("Novo inimigo!")  
  
clock.schedule_interval(criar_inimigo, 2.0)
```

Isso executa criar_inimigo aproximadamente a cada 2 segundos enquanto o jogo estiver rodando.

21. Estados do jogo

Um jogo costuma ter estados como jogando, game over e vitória. Uma variável simples pode controlar isso.

```
estado = "jogando"

def update():
    global estado

    if estado != "jogando":
        return

    # lógica do jogo aqui

    if player.colliderect(inimigo):
        estado = "game_over"

def draw():
    screen.clear()

    if estado == "jogando":
        player.draw()
        inimigo.draw()
    elif estado == "game_over":
        screen.draw.text("GAME OVER", center=(400, 300), fontsize=70, color="red")
```

Parte 4 - Projeto final: Caça às Moedas

Agora vamos juntar movimento, Actor, colisão, pontuação, inimigo e estado de jogo. O objetivo é coletar moedas sem encostar no inimigo.

Preparação das pastas

```
meu_jogo/  
├─ jogo.py  
├─ images/  
│   ├─ player.png  
│   ├─ enemy.png  
│   └─ coin.png  
└─ sounds/  
    └─ collect.wav
```

Código completo

```
import random  
  
WIDTH = 800  
HEIGHT = 600  
  
player = Actor("player", (400, 300))  
inimigo = Actor("enemy", (100, 100))  
moeda = Actor("coin", (650, 450))  
  
velocidade = 4  
pontos = 0  
estado = "jogando"  
  
def mover_moeda():  
    moeda.x = random.randint(40, WIDTH - 40)  
    moeda.y = random.randint(40, HEIGHT - 40)  
  
def update():  
    global pontos, estado  
  
    if estado != "jogando":  
        return  
  
    if keyboard.left:  
        player.x -= velocidade
```

```

if keyboard.right:
    player.x += velocidade
if keyboard.up:
    player.y -= velocidade
if keyboard.down:
    player.y += velocidade

player.x = max(player.width / 2, min(WIDTH - player.width / 2, player.x))
player.y = max(player.height / 2, min(HEIGHT - player.height / 2, player.y))

# inimigo persegue o jogador
if inimigo.x < player.x:
    inimigo.x += 1.5
elif inimigo.x > player.x:
    inimigo.x -= 1.5

if inimigo.y < player.y:
    inimigo.y += 1.5
elif inimigo.y > player.y:
    inimigo.y -= 1.5

if player.colliderect(moeda):
    pontos += 1
    sounds.collect.play()
    mover_moeda()

if player.colliderect(inimigo):
    estado = "game_over"

def on_key_down(key):
    global estado, pontos

    if estado == "game_over" and key == keys.R:
        pontos = 0
        player.pos = (400, 300)
        inimigo.pos = (100, 100)
        mover_moeda()
        estado = "jogando"

def draw():
    screen.fill((20, 24, 32))

    if estado == "jogando":
        player.draw()
        inimigo.draw()
        moeda.draw()
        screen.draw.text(f"Pontos: {pontos}", (20, 20), fontsize=36, color="white")

```



```
else:  
    screen.draw.text("GAME OVER", center=(WIDTH / 2, 240), fontsize=72, color="red")  
    screen.draw.text(f"Pontos: {pontos}", center=(WIDTH / 2, 320), fontsize=42, color="white")  
    screen.draw.text("Pressione R para reiniciar", center=(WIDTH / 2, 390), fontsize=32, color="yellow")
```

Entendendo o projeto

Recursos: player, inimigo e moeda são Actors criados com imagens da pasta images.

Movimento: O jogador usa keyboard; o inimigo compara suas coordenadas com as do jogador para persegui-lo.

Colisão: colliderect() detecta quando o jogador toca a moeda ou o inimigo.

Pontuação: A variável pontos aumenta e a moeda é reposicionada aleatoriamente.

Estado: Quando ocorre colisão com o inimigo, estado passa para game_over e update() deixa de executar a lógica normal.

Reinício: on_key_down() permite reiniciar pressionando R.

Desafios para melhorar o jogo

- Faça o inimigo ficar mais rápido a cada 5 pontos.
- Adicione uma segunda moeda especial que vale 5 pontos.
- Crie três vidas antes do Game Over.
- Adicione um cronômetro de 60 segundos.
- Crie uma tela inicial com “Pressione ENTER para jogar”.
- Adicione vários inimigos usando uma lista.
- Crie fases: a cada 10 pontos, aumente a dificuldade.
- Adicione animação trocando a imagem do Actor enquanto ele se move.

Parte 5 - Como evoluir depois deste guia

Depois de dominar os exercícios e o projeto final, o próximo passo é construir pequenos jogos completos. Projetos curtos ensinam mais do que tentar começar por um RPG enorme.

Sequência recomendada de projetos

Projeto	Conceitos principais
1. Pong	Movimento, colisão, pontuação, velocidade.
2. Space Shooter	Tiros, listas de inimigos, spawn, sons.
3. Endless Runner	Gravidade, pulo, obstáculos e dificuldade progressiva.
4. Top-down Collect Game	Mapa simples, inimigos, itens e estados.
5. Mini RPG	Inventário básico, atributos, combate e múltiplas telas.

Python que vale estudar em seguida

- Dicionários e conjuntos
- Classes e orientação a objetos
- Tratamento de erros com try/except
- Leitura e escrita de arquivos
- Módulos e organização em múltiplos arquivos
- Ambientes virtuais (venv)
- Git e GitHub
- Matemática básica para jogos: vetores, distância e ângulos

Checklist: você está pronto para avançar?

- ☐ Consigo criar e modificar variáveis sem copiar exemplo.
- ☐ Entendo if, elif e else.
- ☐ Consigo usar for e listas.
- ☐ Consigo criar e chamar funções.
- ☐ Entendo a diferença entre draw() e update().
- ☐ Consigo mover um Actor com teclado.
- ☐ Consigo detectar colisão e atualizar pontuação.

- ❑ Consigo criar estado de Game Over e reiniciar o jogo.
- ❑ Quando dá erro, tento interpretar a mensagem antes de procurar a resposta.

Erros comuns de iniciante

Indentação incorreta: Em Python, espaços definem blocos. Mantenha 4 espaços dentro de funções e condições.

Nome de arquivo errado: O nome usado em Actor("player") precisa corresponder a uma imagem player.png, player.jpg etc. na pasta correta.

Executar com python em vez de pgzrun: Para programas Pygame Zero tradicionais, use pgzrun arquivo.py.

Alterar posição em draw(): Use update() para lógica/movimento e draw() para desenhar.

Esquecer global: Se uma função atribui um novo valor a uma variável global simples, pode ser necessário declarar global nome.

Tentar fazer tudo de uma vez: Crie primeiro uma versão mínima funcionando. Depois adicione uma mecânica por vez.

Referências oficiais

Pygame Zero - documentação estável: <https://pygame-zero.readthedocs.io/en/stable/>

Documentação PDF do Pygame Zero 1.2.1:

https://pygame-zero.readthedocs.io/_/downloads/en/1.2.1/pdf/

Python - tutorial oficial: <https://docs.python.org/3/tutorial/>

Como consultar documentação: A documentação não precisa ser lida do começo ao fim. Use-a como referência quando quiser descobrir o que um recurso faz ou quais opções ele possui.

Mensagem final

Programação fica mais fácil quando você transforma cada conceito em algo visível. Em jogos, uma variável pode virar vida, uma condição pode virar Game Over, uma lista pode virar uma horda de inimigos e uma função pode virar um ataque. Continue criando projetos pequenos, terminando cada um e melhorando aos poucos.

Bons estudos e bons jogos!